



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: Архитектура клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: «Разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний»

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Работа представлена к защите _____ (дата)

_____/Миркин К.Л./
(подпись и ф.и.о. студента)

Руководитель: профессор, Мифтахов Эльдар Наилевич

Работа допущена к защите _____ (дата)

_____/Мифтахов Э.Н./
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____/ Мифтахов Эльдар Наилевич, профессор, д.ф.-м.н. _____/

_____/ Ермаков Сергей Романович, старший преподаватель _____/

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

2025 г.

Заведующему кафедрой
инструментального и прикладного
программного обеспечения (ИиППО)
Института информационных технологий (ИТ)
Болбакову Роману Геннадьевичу

От студента Миркина

Кирилла Леонидовича

ФИО

ИКБО-10-23

группа

3 курс

курс

Контакт: 8(916) 485-67-25, mirkin.k.l@edu.mirea.ru

Заявление

Прошу утвердить мне тему курсовой работы по дисциплине «Архитектура клиент-серверных приложений» образовательной программы бакалавриата 09.03.04 (Программная инженерия). Тема: Разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний.

Приложение: лист задания на КР/КП в 2-ух экземплярах на двухстороннем листе (проект)

Подпись студента /Миркин К.Л./
подпись ФИО

Дата _____

Подпись руководителя / проф. Мифтахов Э.Н./
подпись Должность, ФИО

Дата _____

Согласовано _____ /к.т.н., зав. кафедрой ИиППО, Болбаков Р.Г.



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ на выполнение курсовой работы

по дисциплине: Архитектура клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Срок представления к защите: 08.12.2025

Руководитель: д.ф.-м.н., профессор, Мифтахов Эльдар Наилевич

Тема: «Разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний»

Исходные данные: Архитектурные стили и паттерны: клиент-серверная архитектура, многослойная архитектура, API-first, Feature-Sliced Design, Reverse Proxy; языки моделирования и нотации: UML, DFD, ER; используемые технологии: языки программирования – Python TypeScript; фреймворки – FastAPI, React; СУБД – PostgreSQL; инструменты: SQLAlchemy, Vite; открытое ПО: Visual Studio Code, Draw.io, Git, Docker.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Дать описание предметной области, для которой разрабатывается ПО; 2. Определить и сформировать перечень функциональных и нефункциональных требований к ПО; 3. Определить и описать ограничения проекта; 4. Провести обзор архитектурных стилей для выбора наиболее подходящей архитектуры; 5. Разработать набор детализированных диаграмм архитектуры; 6. Изучить и описать популярные паттерны проектирования, которые можно применить для данного проекта; 7. Обосновать выбор технологий для реализации выбранной архитектуры; 8. Разработать прототип приложения с минимально жизнеспособным набором функций (MVP) и развернуть его в соответствующей инфраструктуре.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____ / Болбаков Р. Г. /, «____» _____ 2025 г.

Задание на КР выдал: _____ / Мифтахов Э.Н. /, «____» _____ 2025 г.

Задание на КР получил: _____ / Миркин К.Л. /, «____» _____ 2025 г.

РЕФЕРАТ

Отчёт 48 с., 2 рис., 3 табл., 25 ист.

CLEAN ARCHITECTURE, FASTAPI, REST API, СЕРВЕРНАЯ ЧАСТЬ

Миркин К.Л., Курсовая работа направления подготовки «Программная инженерия» предмета «Бэкенд-разработка» на тему «Серверная часть веб-приложения для управления образовательными курсами»: М. 2026 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО)

Курсовая работа посвящена разработке серверной части веб-приложения для управления образовательными курсами. Выполнен анализ предметной области, сформированы функциональные и нефункциональные требования к системе управления обучением (LMS), проведён обзор существующих решений с построением матрицы отслеживаемости аналогов.

Проведён анализ архитектурных паттернов DDD, Clean Architecture и MVC, выполнен их сравнительный анализ и обоснован выбор Clean Architecture в качестве основного подхода к проектированию серверной части. Разработана детальная архитектура системы на основе Clean Architecture с описанием слоёв API, сервисов, репозиторий и моделей.

Реализована серверная часть на языке Python с использованием фреймворка FastAPI, СУБД PostgreSQL и библиотеки SQLAlchemy. Приведены листинги кода, описана структура REST API, слой базы данных и механизмы аутентификации. Корректность разработанной архитектуры подтверждена реализованным прототипом системы.

СОДЕРЖАНИЕ

Определения, обозначения и сокращения	7
Введение	9
1 Анализ предметной области и требований	11
1.1 Описание предметной области	11
1.2 Функциональные требования	12
1.3 Нефункциональные требования	13
1.4 Ограничения проекта	15
1.5 Анализ существующих LMS-решений	15
1.6 Выводы по главе	18
2 Выбор и обоснование технологий	19
2.1 Анализ архитектурных паттернов	19
2.1.1 Domain-Driven Design (DDD)	19
2.1.2 Clean Architecture	20
2.1.3 Model-View-Controller (MVC)	21
2.2 Сравнительный анализ паттернов	21
2.3 Обоснование выбора технологического стека	23
2.4 Выводы по главе	24
3 Разработка архитектуры приложения на основе Clean Architecture	25
3.1 Применение Clean Architecture к серверной части	25
3.2 Dependency Injection как механизм связывания слоёв	27
3.3 Диаграмма компонентов архитектуры	28
3.4 Проектирование базы данных	30
3.5 Выводы по главе	32
4 Разработка серверной части интернет-ресурса	33
4.1 Реализация REST API на FastAPI	33
4.2 Реализация сервисного слоя	34
4.3 Реализация слоя репозитория	37
4.4 Реализация слоя безопасности	39
4.5 Конфигурация и запуск приложения	40
4.6 Интеграция с realtime-чатом	42
4.7 Выводы по главе	43

Заключение	44
Список использованных источников	46
[	

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

]

В настоящем отчёте применяют следующие определения, обозначения и сокращения.

API (Application Programming Interface)	– программный интерфейс, определяющий способы взаимодействия компонентов программного обеспечения.
Argon2	– алгоритм хеширования паролей, устойчивый к атакам с использованием GPU и ASIC.
Clean Architecture	– архитектурный паттерн, предложенный Робертом Мартином, обеспечивающий независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов.
DDD (Domain-Driven Design)	– подход к проектированию программного обеспечения, ориентированный на моделирование предметной области и использование единого языка (Ubiquitous Language).
Dependency Injection (DI)	– паттерн проектирования, при котором зависимости компонентов передаются извне, а не создаются внутри компонента.
DTO (Data Transfer Object)	– объект передачи данных, используемый для обмена структурированной информацией между слоями приложения.
ER-диаграмма (Entity-Relationship Diagram)	– диаграмма «сущность–связь», используемая для моделирования структуры данных и отношений между сущностями.
FastAPI	– современный веб-фреймворк для Python, ориентированный на создание REST API с поддержкой асинхронности, автоматической валидации данных и документирования.
HTTP (HyperText Transfer Protocol)	– протокол передачи данных между клиентом и сервером в веб-среде.

JWT (JSON Web Token)	– компактный и самодостаточный формат для передачи данных в виде JSON-объекта, используемый для аутентификации.
LMS (Learning Management System)	– система управления обучением, предназначенная для организации образовательного процесса, хранения материалов и контроля результатов.
MVC (Model-View-Controller)	– архитектурный паттерн, разделяющий приложение на три взаимосвязанных компонента: модель, представление и контроллер.
ORM (Object-Relational Mapping)	– технология отображения объектных структур в реляционные базы данных.
Repository	– паттерн, инкапсулирующий логику доступа к данным и предоставляющий интерфейс для операций с сущностями.
REST (Representational State Transfer)	– архитектурный стиль взаимодействия распределённых систем, использующий принципы унифицированного интерфейса и обмена представлениями ресурсов.
Service Layer	– слой бизнес-логики приложения, обрабатывающий прикладные операции и обращающийся к репозиториям.
SQL (Structured Query Language)	– язык запросов, используемый для управления реляционными базами данных.
SQLAlchemy	– библиотека ORM и инструмент работы с реляционными базами данных в Python.
СУБД	– система управления базами данных.
Фреймворк	– программный каркас, упрощающий разработку приложений и задающий архитектурные принципы.

ВВЕДЕНИЕ

Разработка серверной части современных веб-приложений представляет собой одну из ключевых дисциплин в области программной инженерии. Серверная часть (backend) отвечает за обработку бизнес-логики, хранение и предоставление данных, аутентификацию пользователей, обеспечение безопасности и масштабируемости системы. Качество архитектурных решений, принятых на этапе проектирования серверной части, напрямую определяет такие характеристики итогового продукта, как производительность, надёжность, сопровождаемость и возможность дальнейшего расширения функциональности [1].

Одной из актуальных областей применения backend-разработки являются системы управления обучением (Learning Management Systems, LMS) [2]. Такие системы обеспечивают организацию учебного процесса в цифровой среде: управление курсами и учебными материалами, проведение тестирования, отслеживание успеваемости и коммуникацию между участниками образовательного процесса. С ростом популярности дистанционного и смешанного форматов обучения возрастают требования к архитектуре подобных систем, что делает задачу проектирования и реализации их серверной части особенно значимой.

Актуальность выбранной темы определяется необходимостью разработки серверной части современной LMS-системы с применением актуальных архитектурных паттернов и технологического стека. Особое внимание уделяется Clean Architecture — подходу, обеспечивающему независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов. Применение данного паттерна позволяет создать систему, устойчивую к изменениям требований и удобную для сопровождения [3].

Цель работы — разработка серверной части веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture с использованием современных технологий и инструментов.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области и определить требования к серверной части LMS-системы;
- провести обзор и сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC, применимых к разработке серверной части;
- обосновать выбор технологического стека (FastAPI, PostgreSQL, SQLAlchemy) для реализации серверной части;
- разработать архитектуру серверной части на основе Clean Architecture с детальным описанием слоёв;
- реализовать серверную часть приложения, включая REST API, слой бизнес-логики и слой доступа к данным;
- привести листинги кода, демонстрирующие применение выбранных архитектурных решений.

Объект исследования — процессы организации и сопровождения онлайн-обучения.

Предмет исследования — архитектурные и технологические решения при разработке серверной части системы управления обучением с применением Clean Architecture.

1 Анализ предметной области и требований

1.1 Описание предметной области

Система управления онлайн-обучением (LMS) — это программная платформа, предназначенная для организации, сопровождения и контроля учебного процесса. Она позволяет предоставлять пользователям (студентам, слушателям) доступ к учебным курсам, учебным материалам, тестам и заданиям, а преподавателям — управлять контентом, заданиями, проверкой и оценкой знаний. Кроме того, администраторы системы обеспечивают управление пользователями, ролями, доступами, данными и общей инфраструктурой платформы.

Предметная область разрабатываемой системы включает несколько ключевых компонентов:

- **учебные курсы**, содержащие теоретические материалы, практические задания, тесты и мультимедийные ресурсы;
- **пользователей** с различными ролями, среди которых администраторы, преподаватели и обучающиеся;
- **механизмы тестирования и оценки знаний**, обеспечивающие контроль усвоения материала;
- **хранилище данных**, в котором содержится информация о пользователях, курсах, результатах тестирования и связанных учебных артефактах.

Типичные сценарии использования LMS охватывают регистрацию и авторизацию пользователей, назначение ролей, управление курсами, загрузку обучающих материалов, формирование тестов, прохождение оценочных заданий, сохранение результатов и формирование отчётности. Эти процессы характерны для современных платформ дистанционного обучения, что подтверждается исследованиями, в которых рассматриваются структура и функциональные возможности LMS [2]. При этом в основе построения подобных систем лежат общие принципы программной инженерии, включая модульность, разделение ответственности и изоляцию слоёв [4].

Таким образом, предметная область проекта представляет собой совокупность сущностей и процессов, обеспечивающих полноценную

организацию учебной деятельности в цифровой среде: управление курсами и контентом, взаимодействие между пользователями, проведение тестирования и хранение данных об успеваемости. Данное описание задаёт основу для последующего формирования требований к системе и проектирования её архитектуры.

1.2 Функциональные требования

Функциональные требования определяют, какие возможности должна предоставлять система пользователям для обеспечения полноценной организации учебного процесса. При формировании функциональных требований учитывается контекст реального использования образовательных информационных систем в крупных городах. Согласно аналитическим данным, публикуемым Аналитическим центром города Москвы [5], цифровые образовательные сервисы применяются в масштабах, охватывающих значительное число обучающихся и преподавателей. В рамках данной работы система рассматривается как платформа, предназначенная для централизованной поддержки учебного процесса в условиях крупной городской образовательной среды.

На основе анализа предметной области можно выделить следующие ключевые функции платформы:

- **Управление учебными курсами.** Преподаватель должен иметь возможность создавать курсы, формировать их структуру, добавлять учебные материалы и обновлять содержимое. Обучающемуся предоставляется доступ к назначенным курсам и материалам в установленной последовательности.

- **Работа с пользователями и ролями.** Система должна поддерживать регистрацию, аутентификацию и разграничение прав доступа. Разные категории пользователей — администраторы, преподаватели и обучающиеся — получают доступ к соответствующим разделам и действиям.

- **Проведение тестирования и фиксация результатов.** Преподаватель формирует тесты, настраивает параметры прохождения и оценивания. Обучающийся проходит тесты, а система сохраняет результаты попыток для последующего анализа.

– Отображение прогресса и статистики. Платформа должна предоставлять информацию об успеваемости: результаты тестов, динамику выполнения заданий и общие показатели по курсу.

– **Информационное взаимодействие.** Пользователи должны получать уведомления о новых материалах, тестах, изменениях в заданиях и других событиях курса.

– **Администрирование системы.** Администратор управляет учетными записями, ролями, принадлежностью к группам, а также общей структурой курсов и параметрами системы.

Перечисленные функции отражают минимально необходимый объём возможностей, который должна обеспечивать разрабатываемая система. Они формируют основу архитектуры платформы и служат отправной точкой для определения нефункциональных требований, ограничений и дальнейшего проектирования.

1.3 Нефункциональные требования

Нефункциональные требования определяют эксплуатационные характеристики системы и ограничения, при которых должны корректно выполняться функциональные возможности. Эти требования оказывают влияние на архитектуру системы, выбор технологий и способы развертывания программного обеспечения [6]. Требования к структуре и оформлению отчётной документации по проекту регламентируются ГОСТ 7.32-2017 [7].

В рамках данной работы система рассматривается как платформа, ориентированная на эксплуатацию в условиях городской образовательной среды с числом зарегистрированных пользователей до 100 000 человек. Количественные значения требований определены исходя из типовых сценариев использования образовательных платформ: массовый одновременный доступ в учебные часы, работа с мультимедийными материалами и регулярное тестирование обучающихся. В таблице 1 приведены основные нефункциональные требования к разрабатываемой системе управления онлайн-обучением.

Таблица 1 – Нефункциональные требования к системе

Критерий	Метрика	Значение	Обоснование
Производительность	Время отклика на действие пользователя	0–1000 мс	Отклик до 100 мс воспринимается как мгновенный, 100–1000 мс соответствует естественным операциям взаимодействия.
	Время загрузки интерфейса	≤ 3 с	Целевое значение, обеспечивающее комфортное взаимодействие при регулярной навигации между уроками и заданиями.
Нагрузка	Пиковая одновременная активность	5 000–10 000 пользователей	Для городской аудитории 100000 зарегистрированных пользователей принимается пиковая активность 10% в учебные часы.
	Интенсивность запросов к API, RPS	500–1000 RPS	10 000 активных пользователей, 1 действие в минуту и 6 API-вызовов на действие. Тогда RPS равен $10000 \cdot (6/60) = 1000$ RPS.
Объём данных	Средний объём данных на 1 урок	0,85–0,90 ГБ	Видеоматериал урока длительностью 10 минут в HD ≈ 850 МБ, а также дополнительные файлы и иллюстрации 20–50 МБ.
	Общий объём учебных материалов	10–20 ТБ	$10\,000 \text{ уроков} \times 0,88 \text{ ГБ} \approx 8,8 \text{ ТБ}$. А также добавляется запас для учёта версионирования материалов, обновлений и служебных данных
	Объём данных результатов обучения	200–300 ГБ за 5 лет	Расчёт: $100 \sim 000 \times 200 \times 2 \text{ КБ} \approx 40 \text{ ГБ/год}$. За 5 лет: $\approx 200 \text{ ГБ}$.
Надёжность	Доступность системы	$\geq 99,5 \%$	Система должна быть доступна в учебные часы и периоды контрольных мероприятий.
Совместимость	Поддержка браузеров	Chrome, Firefox, Edge, Safari	Система должна корректно работать в актуальных версиях основных браузеров в школьной и домашней среде.

Сформулированные нефункциональные требования задают целевые свойства разрабатываемой системы и определяют рамки, в которых должны приниматься архитектурные решения. В дальнейшем они будут учитываться при выборе архитектурных стилей, паттернов проектирования и технологий реализации.

1.4 Ограничения проекта

Проект разрабатывается в условиях ограниченных ресурсов и выполняется одним исполнителем, что определяет необходимость выбора архитектурных решений, доступных по сложности реализации в установленные сроки. Масштаб системы ориентирован на ограниченное число пользователей, характерное для учебных платформ малого и среднего размера, что позволяет не учитывать требования, предъявляемые к высоконагруженным промышленным системам.

Доступные технологии определяются уровнем подготовки разработчика и учебным контекстом проекта, что ограничивает использование сложных инфраструктурных решений и узкоспециализированных инструментов. Архитектура должна оставаться достаточно компактной, чтобы обеспечить возможность реализации основных компонентов без распределения задач между несколькими командами.

Сроки выполнения проекта определяются рамками курсовой работы и включают этапы анализа предметной области, проектирования архитектуры, разработки минимально жизнеспособного прототипа и проведения его тестирования. Это предопределяет необходимость сосредоточиться на ключевых функциях системы и обеспечить приоритетное выполнение базовых механизмов, связанных с управлением курсами, пользователями и тестированием.

1.5 Анализ существующих LMS-решений

Для обоснования функциональных требований и выявления конкурентных преимуществ разрабатываемой системы проведён анализ существующих LMS-платформ, представленных на рынке образовательных технологий. Рассмотрены следующие решения: Moodle, Canvas LMS и

Google Classroom. Каждая из платформ имеет свою целевую аудиторию, архитектурные особенности и набор функциональных возможностей [8].

Moodle — одна из старейших и наиболее распространённых LMS с открытым исходным кодом, предоставляющая широкий набор модулей для управления курсами, тестирования, оценивания и совместной работы. Архитектура Moodle основана на классической трёхслойной модели и использует PHP, MySQL/PostgreSQL. К преимуществам платформы относятся гибкость настройки и обширное сообщество разработчиков, к недостаткам — сложность развёртывания и устаревший пользовательский интерфейс.

Canvas LMS — современная облачная LMS-платформа от компании Instructure, ориентированная на удобство использования и интеграцию с внешними сервисами. Canvas построен на Ruby on Rails и PostgreSQL, предоставляет REST API и поддерживает стандарты LTI (Learning Tools Interoperability). Преимуществами Canvas являются высокая производительность, современный интерфейс и хорошая документация API, однако платформа является проприетарной и требует платной подписки.

Google Classroom — бесплатная платформа от Google, интегрированная с экосистемой Google Workspace for Education. Архитектура базируется на облачной инфраструктуре Google, что обеспечивает высокую масштабируемость, но ограничивает возможности кастомизации. Google Classroom предоставляет минимальный базовый функционал для управления курсами и заданиями, ориентируясь на простоту использования.

На основе сравнительного анализа выделенных платформ сформирована матрица отслеживаемости, представленная в таблице 2. Матрица позволяет сопоставить функциональные возможности рассматриваемых решений и определить требования к разрабатываемой системе.

Таблица 2 – Матрица отслеживаемости аналогов LMS-систем

Функциональная возможность	Moodle	Canvas LMS	Google Classroom	Разрабатываемая система
Управление курсами и уроками	Полная поддержка	Полная поддержка	Базовая поддержка	Полная поддержка
Создание и проверка ДЗ	Полная поддержка	Полная поддержка	Базовая поддержка	Полная поддержка
Тестирование и оценка знаний	Полная поддержка	Полная поддержка	Ограниченная	Полная поддержка
Чат класса и уведомления	Встроенный чат + форумы	Встроенный чат	Google Chat	Веб-сокет чат в реальном времени
REST API	Поддерживается	Полноценный REST API	Ограниченный	Полноценный REST API (FastAPI)
Асинхронная обработка	Нет (синхронный PHP)	Частично	Облачная инфраструктура	Полностью асинхронный (asyncio)
Clean Architecture	Нет (MVC-подобная)	Нет	Нет	Clean Architecture
Открытый исходный код	GPL v3	Нет	Нет	MIT
Установка на собственный сервер	Требуется	Облачная + on-premise	Только облачная	Docker-контейнеризация
Time-to-развёртывание MVP	2–4 недели	1–2 недели	1 день	1–2 недели

Проведённый анализ показал, что существующие решения обладают рядом ограничений: Moodle требует значительных усилий по развёртыванию и имеет устаревшую архитектуру; Canvas LMS является проприетарным решением с высокой стоимостью подписки; Google Classroom не предоставляет возможностей для глубокой кастомизации и расширения функциональности. Разрабатываемая система призвана компенсировать указанные недостатки за счёт применения Clean Architecture, современного асинхронного стека FastAPI и PostgreSQL, а также открытой лицензии MIT, позволяющей свободное использование и модификацию. Инфраструктура развёртывания основана на Docker-контейнеризации, что обеспечивает воспроизводимость окружения и упрощает масштабирование [9]. Принципы

организации REST API, используемые в системе, соответствуют стандартным подходам к проектированию интерфейсов веб-приложений [10,11].

1.6 Выводы по главе

В данной главе проведён анализ предметной области систем управления обучением (LMS). Определено, что разрабатываемая система должна обеспечивать управление курсами и уроками, проведение тестирования, отслеживание успеваемости, чат и администрирование. Сформулированы функциональные и нефункциональные требования к серверной части, включая требования к производительности, нагрузке, объёму данных и надёжности. Определены ограничения проекта, связанные с ресурсами и сроками выполнения.

Выполнен анализ существующих LMS-решений: Moodle, Canvas LMS и Google Classroom. Сформирована матрица отслеживаемости, позволившая сравнить функциональные возможности рассматриваемых платформ. На основе анализа выявлены преимущества разрабатываемой системы: лёгкая архитектура на основе Clean Architecture, современный асинхронный стек FastAPI и PostgreSQL, открытая лицензия MIT и Docker-контейнеризация. Полученные результаты формируют основу для выбора технологий и проектирования архитектуры серверной части.

2 Выбор и обоснование технологий

2.1 Анализ архитектурных паттернов

Архитектурные паттерны проектирования определяют способ организации программного кода, распределение ответственности между компонентами системы и характер взаимодействия между ними. Выбор подходящего паттерна является критическим этапом разработки серверной части, поскольку от него зависят такие характеристики системы, как тестируемость, сопровождаемость, расширяемость и независимость от конкретных фреймворков [1].

В рамках данной работы рассматриваются три ключевых архитектурных паттерна, применимых к разработке серверной части: Domain-Driven Design (DDD), Clean Architecture (предложенная Робертом Мартином) и Model-View-Controller (MVC) [3,12].

2.1.1 Domain-Driven Design (DDD)

Domain-Driven Design — подход к проектированию программного обеспечения, предложенный Эриком Эвансом, в основе которого лежит моделирование предметной области. Ключевой идеей DDD является создание единой модели предметной области (domain model), которая отражает реальные бизнес-процессы и правила, и использование единого языка (Ubiquitous Language), понятного как разработчикам, так и экспертам предметной области [12].

Основные концепции DDD включают:

- **Сущности (Entities)** — объекты, обладающие уникальным идентификатором и изменяемым состоянием (например, пользователь или курс).
- **Объекты-значения (Value Objects)** — неизменяемые объекты, характеризующиеся своими атрибутами (например, адрес электронной почты).
- **Агрегаты (Aggregates)** — группы связанных сущностей и объектов-значений, рассматриваемые как единое целое.
- **Репозитории (Repositories)** — механизмы получения и хранения агрегатов.

– **Доменные сервисы (Domain Services)** — операции, не уместяющиеся в рамки одной сущности.

DDD обеспечивает глубокое соответствие программной модели бизнес-требованиям, однако требует активного участия экспертов предметной области и значительных усилий на этапе моделирования, что может быть избыточным для проектов с ограниченными ресурсами.

2.1.2 Clean Architecture

Clean Architecture — архитектурный паттерн, предложенный Робертом Мартином (Robert C. Martin, «Uncle Bob») в 2012 году и детально описанный в книге «Чистая архитектура» [3]. Основная идея паттерна заключается в организации кода в виде концентрических слоёв, где внутренние слои содержат бизнес-логику и не зависят от внешних слоёв — фреймворков, баз данных и пользовательских интерфейсов.

Clean Architecture базируется на следующих принципах:

– **Независимость от фреймворков** — бизнес-логика не должна зависеть от используемого веб-фреймворка, что позволяет менять технологический стек без изменения ядра системы.

– **Тестируемость** — бизнес-правила могут быть протестированы без участия внешних компонентов, таких как база данных или веб-сервер.

– **Независимость от UI** — пользовательский интерфейс может изменяться без влияния на бизнес-логику.

– **Независимость от базы данных** — выбор СУБД может быть изменён без изменения бизнес-правил.

Стандартная структура Clean Architecture в контексте серверной части включает четыре слоя:

1) **Controllers / API Layer** — обработка входящих HTTP-запросов, валидация входных данных, маршрутизация.

2) **Use Cases / Service Layer** — реализация бизнес-сценариев, координация между репозиториями и внешними сервисами.

3) **Interface Adapters / Repository Layer** — абстракция доступа к данным, преобразование между форматами ORM и DTO.

4) **Frameworks & Drivers / Models Layer** — реализация инфраструктурных деталей: ORM-модели, HTTP-клиенты, драйверы базы данных.

Правило зависимостей (Dependency Rule) гласит: зависимости всегда направлены внутрь, то есть внешние слои могут зависеть от внутренних, но не наоборот.

2.1.3 Model-View-Controller (MVC)

Model-View-Controller — классический архитектурный паттерн, разделяющий приложение на три компонента:

- **Model (Модель)** — данные и бизнес-логика приложения.
- **View (Представление)** — отображение данных пользователю.
- **Controller (Контроллер)** — обработка пользовательского ввода и управление потоком данных.

MVC широко применяется в веб-фреймворках (Django, Ruby on Rails, ASP.NET MVC) и обеспечивает чёткое разделение ответственности между логикой представления и бизнес-логикой. Однако в контексте серверной разработки MVC имеет недостаток: жёсткая связь между контроллерами и фреймворком, а также смешение бизнес-логики и инфраструктурного кода.

2.2 Сравнительный анализ паттернов

Для объективного выбора архитектурного паттерна проведено сравнение DDD, Clean Architecture и MVC по следующим критериям: сложность внедрения, тестируемость, независимость от фреймворков, масштабируемость, применимость в условиях ограниченных ресурсов и соответствие требованиям REST API. Результаты сравнения представлены в таблице 3.

Таблица 3 – Сравнительный анализ архитектурных паттернов

Критерий	DDD	Clean Architecture	MVC
Сложность внедрения	Высокая	Средняя	Низкая
Тестируемость бизнес-логики	Высокая	Высокая	Низкая
Независимость от фреймворков	Средняя (зависит от реализации)	Полная	Низкая (контроллер привязан к фреймворку)
Независимость от БД	Высокая (через репозитории)	Полная (через репозитории)	Низкая
Разделение ответственности	Чёткое (слои + агрегаты)	Чёткое (концентрические слои)	Умеренное (Model/View/Controller)
Применимость для REST API	Хорошая	Отличная	Умеренная
Требования к команде	Высокие (необходим domain expert)	Средние	Низкие
Применимость в учебном проекте	Избыточна	Оптимальна	Упрощённая, но хуже тестируема

На основе проведённого сравнительного анализа в качестве основного архитектурного паттерна выбрана Clean Architecture. Данный выбор обосновывается следующими факторами:

- Clean Architecture обеспечивает независимость бизнес-логики от фреймворков, что позволяет заменять технологические компоненты без переписывания ядра системы.

- Паттерн обеспечивает высокую тестируемость: бизнес-правила могут тестироваться изолированно, без необходимости поднимать базу данных или HTTP-сервер.

- Clean Architecture хорошо сочетается с REST-подходом, где API-контроллеры выступают внешним слоем, а бизнес-логика остаётся независимой.

- По сравнению с DDD, Clean Architecture требует меньше усилий на начальном этапе и не требует обязательного участия экспертов предметной области, что важно в условиях ограниченных ресурсов учебного проекта.

– По сравнению с MVC, Clean Architecture обеспечивает лучшее разделение ответственности и более высокую тестируемость, хотя и требует больше начальных усилий по структурированию кода.

2.3 Обоснование выбора технологического стека

После выбора архитектурного паттерна необходимо обосновать технологический стек, обеспечивающий его реализацию. Для серверной части системы управления обучением выбраны следующие технологии.

Python 3.12+. Язык Python является одним из наиболее популярных языков для веб-разработки благодаря читаемому синтаксису, богатой экосистеме библиотек и активному сообществу. Версия 3.12+ предоставляет улучшенную поддержку асинхронного программирования, более строгую систему типов через механизм аннотаций (type hints) и оптимизации производительности [13]. Python хорошо подходит для реализации Clean Architecture благодаря поддержке абстрактных классов, протоколов (typing.Protocol) и dependency injection [8].

FastAPI. В качестве основного веб-фреймворка выбран FastAPI, который является одним из наиболее быстрорастущих Python-фреймворков для создания REST API [14,15]. По данным опроса разработчиков Stack Overflow 2025, FastAPI входит в число наиболее популярных веб-технологий, что подтверждает его актуальность и востребованность [16]. FastAPI обеспечивает:

- высокую производительность за счёт асинхронности на базе asyncio;
- автоматическую валидацию данных через Pydantic (версия 2);
- автоматическую генерацию документации OpenAPI/Swagger;
- встроенную поддержку Dependency Injection для реализации принципов Clean Architecture;
- строгую типизацию на уровне API.

PostgreSQL. Реляционная СУБД PostgreSQL выбрана благодаря поддержке транзакционной целостности ACID, расширенных типов данных (JSONB, UUID, массивы) и развитой системе индексов [17]. PostgreSQL является отраслевым стандартом для веб-приложений среднего и крупного масштаба, обеспечивая высокую надёжность и производительность [18].

SQLAlchemy (asyncio). Для работы с базой данных используется SQLAlchemy — наиболее зрелая ORM для Python с поддержкой асинхронного драйвера (через `asyncpg`) [19]. Для управления миграциями схемы базы данных применяется инструмент Alembic, обеспечивающий версионирование изменений [20]. SQLAlchemy поддерживает паттерны Data Mapper и Repository, что полностью соответствует принципам Clean Architecture и обеспечивает независимость бизнес-логики от конкретной СУБД [6].

React + TypeScript. Для клиентской части выбран React 18 с TypeScript, что обеспечивает строгую типизацию и предсказуемость кода [21]. Архитектура клиентской части построена на принципах Feature-Sliced Design (FSD) [22], что согласуется с разделением ответственности, принятым в Clean Architecture.

Выбранный технологический стек соответствует всем критериям, определённым на этапе анализа требований: поддержка многослойной архитектуры, эффективная работа с данными, соответствие современным стандартам REST API, возможность масштабирования и актуальность в индустрии.

2.4 Выводы по главе

В данной главе проведён анализ и сравнительное исследование трёх архитектурных паттернов: Domain-Driven Design, Clean Architecture и Model-View-Controller. На основе сравнения по восьми критериям обоснован выбор Clean Architecture как основного паттерна проектирования серверной части. Clean Architecture обеспечивает оптимальный баланс между тестируемостью, независимостью от фреймворков и сложностью внедрения, что делает её наилучшим выбором для разрабатываемой системы.

Определён технологический стек проекта: Python 3.12+, FastAPI, PostgreSQL, SQLAlchemy для серверной части и React + TypeScript для клиентской части. Выбранные технологии соответствуют современным стандартам разработки и обеспечивают эффективную реализацию Clean Architecture.

3 Разработка архитектуры приложения на основе Clean Architecture

3.1 Применение Clean Architecture к серверной части

На основе выбранного архитектурного паттерна спроектирована архитектура серверной части системы управления обучением. Архитектура организована в соответствии с принципом разделения на концентрические слои, где каждый слой имеет строго определённую зону ответственности, а зависимости направлены внутрь — от внешних инфраструктурных компонентов к внутренней бизнес-логике [3]. Принцип многослойной организации, общие подходы к разделению логики и управлению зависимостями в сложных программных системах также подробно рассматриваются в литературе по архитектуре информационных систем [23].

В рамках разработанной архитектуры выделены следующие слои:

- **API Layer (Controllers)** — внешний слой, отвечающий за приём HTTP-запросов, валидацию входных данных через Pydantic-схемы и формирование HTTP-ответов. Реализован с использованием FastAPI-роутеров. Не содержит бизнес-логики, только делегирование вызовов сервисному слою.

- **Service Layer (Use Cases)** — слой бизнес-логики, реализующий сценарии использования системы. Каждый сервис инкапсулирует определённую функциональную область: аутентификация, управление классами, работа с уроками, домашними заданиями, тестирование и т.д. Сервисы не имеют прямых зависимостей от HTTP-контекста или базы данных, используя абстракции репозитория.

- **Repository Layer (Interface Adapters)** — слой доступа к данным, реализующий паттерн Repository. Предоставляет интерфейсы для выполнения операций CRUD и специфических запросов, скрывая детали реализации SQLAlchemy ORM.

- **Models Layer (Frameworks & Drivers)** — самый внутренний инфраструктурный слой, содержащий определения ORM-моделей, конфигурацию базы данных, настройки миграций и внешние интеграции (Redis для realtime-чата).

Правило зависимостей (Dependency Rule) соблюдается следующим образом: API-слой зависит от Service-слоя, Service-слой зависит от Repository-

слоя, Repository-слой использует Models. Модели не зависят ни от какого другого слоя проекта. Обратные зависимости исключены — ни один внутренний слой не импортирует компоненты внешних слоёв.

Данная организация кода реализована в структуре директорий серверного приложения:

```
app/
├─ main.py                # Entry point
├─ api/                   # API Layer
│  └─ v1/                 # REST endpoints
│     └─ dependencies.py  # Dependency Injection
│     └─ errors.py        # Обработка ошибок
│     └─ openapi.py       # Настройка OpenAPI
│     └─ middleware/      # Middleware (CORS, request_id)
├─ services/              # Service Layer (бизнес-логика)
│  └─ auth.py             # Аутентификация
│  └─ classroom.py        # Управление классами
│  └─ lesson.py           # Управление уроками
│  └─ homework.py         # Домашние задания
│  └─ problem.py          # База задач
│  └─ testing.py          # Приём ответов, проверка
│  └─ result.py           # Статистика и прогресс
│  └─ theory.py           # Теоретические материалы
│  └─ chat.py             # Чат класса
├─ repositories/          # Repository Layer (доступ к данным)
│  └─ base.py             # Базовый репозиторий (CRUD)
│  └─ user.py             # Пользователи
│  └─ classroom.py        # Классы
│  └─ lesson.py           # Уроки
│  └─ homework.py         # Домашние задания
│  └─ communication.py    # Чат и сообщения
├─ models/                # Models Layer (ORM)
│  └─ users.py            # Таблицы пользователей
│  └─ classes.py          # Учебные классы
│  └─ lessons.py          # Уроки
│  └─ homework.py         # Домашние задания
│  └─ problems.py         # Задачи
│  └─ theory.py           # Теоретические материалы
```

```

|   └─ files.py           # Файлы
|   └─ communication.py  # Сообщения чата
└─ schemas/              # Pydantic DTO
└─ core/                 # Конфигурация и утилиты
|   └─ config.py         # Настройки (.env)
|   └─ security.py       # Argon2 + JWT
|   └─ pagination.py     # Пагинация
└─ db/                   # База данных
|   └─ session.py        # AsyncSession
|   └─ url.py            # URL-хелперы
└─ domain/               # Доменные ошибки
└─ realtime/             # WebSocket и Redis Pub/Sub
└─ scripts/              # Утилиты (seed, export OpenAPI)

```

Таким образом, структура директорий непосредственно отражает выбранную Clean Architecture. Каждый слой размещён в отдельном пакете, и зависимости строго направлены от внешних пакетов к внутренним.

3.2 Dependency Injection как механизм связывания слоёв

Ключевым механизмом, обеспечивающим соблюдение правила зависимостей в разработанной архитектуре, является Dependency Injection (DI). FastAPI предоставляет встроенную поддержку DI через механизм зависимостей (Depends), который используется для связывания слоёв без создания жёстких зависимостей между ними.

На уровне API создаются функции-фабрики, которые конструируют сервисы с необходимыми зависимостями. Пример реализации DI для аутентификации приведён в листинге 1.

Листинг 1 – Реализация Dependency Injection для сервисов аутентификации

```

# app/api/dependencies.py
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db
from app.services.auth import AuthService
from app.repositories.user import UserRepository

async def get_auth_service(
    db: AsyncSession = Depends(get_db),
) -> AuthService:
    return AuthService(db)

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: AsyncSession = Depends(get_db),
) -> User:
    """Извлекает и проверяет JWT-токен, возвращает пользователя."""
    payload = decode_access_token(credentials.credentials)
    user_repo = UserRepository(db)
    user = await user_repo.get_by_id(int(payload["sub"]))
    if not user:
        raise HTTPException(status_code=401)
    return user

```

В данном листинге показано, как Dependency Injection позволяет инстанцировать сервисы с их зависимостями (сессия базы данных, репозитории) без создания глобальных переменных или жёстких связей. Каждый тест может подставить свою реализацию репозитория, заменив реальную базу данных на mock.

3.3 Диаграмма компонентов архитектуры

На рисунке 1 представлена диаграмма компонентов архитектуры, отражающая взаимодействие между слоями системы. Внешний слой (API Controller) принимает HTTP-запросы от клиента и делегирует выполнение бизнес-операций соответствующим сервисам. Сервисы используют репозитории для доступа к данным, а репозитории выполняют SQL-запросы через ORM SQLAlchemy к СУБД PostgreSQL.

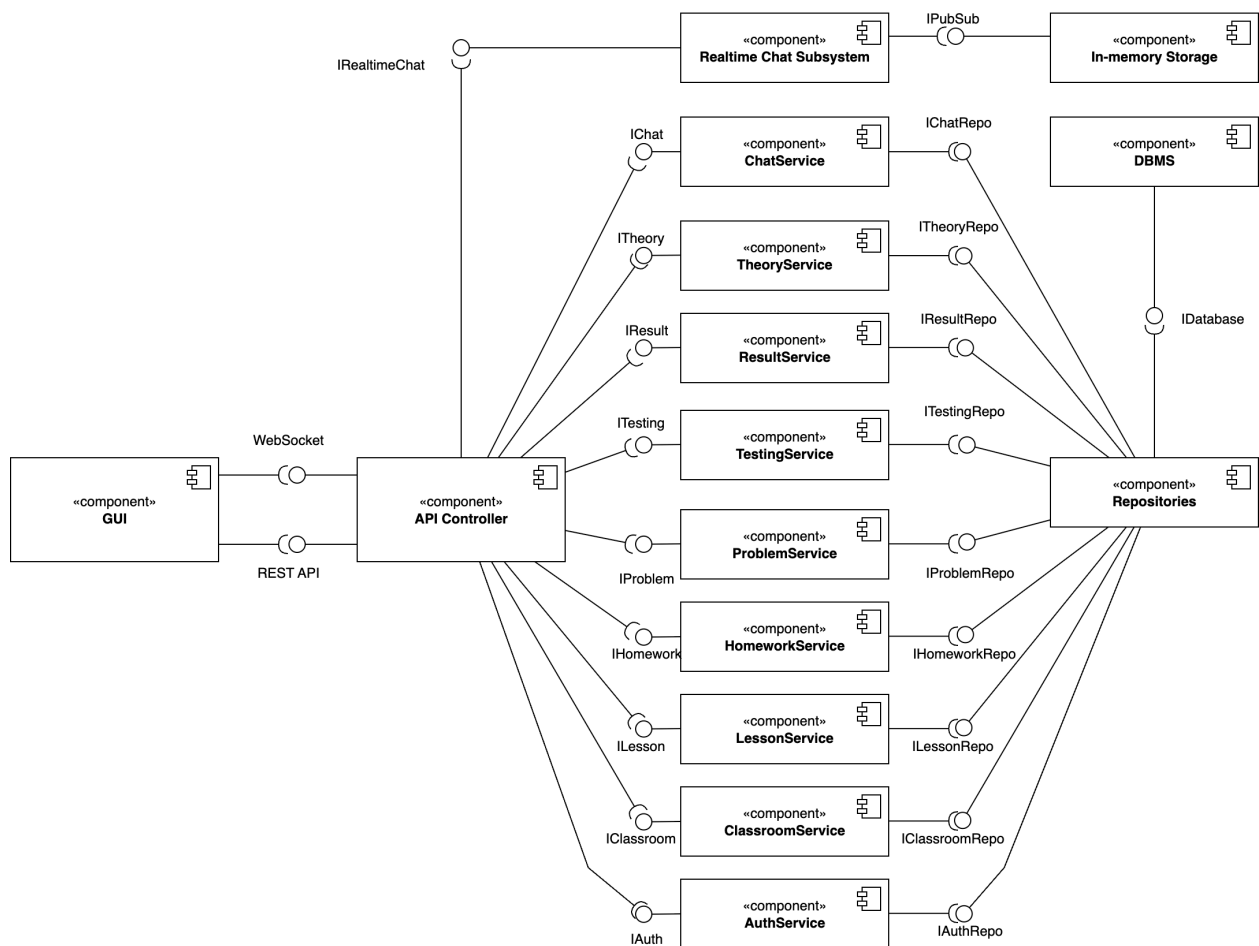


Рисунок 1 – Диаграмма основных компонентов системы, отражающая Clean Architecture

Ключевые компоненты серверной части включают:

- **API Controller** — точка входа для HTTP-запросов, маршрутизация и валидация.
- **AuthService** — аутентификация и сессия (JWT + Argon2).
- **ClassroomService** — управление классами и членством.
- **LessonService** — управление уроками.
- **HomeworkService** — управление домашними заданиями.
- **ProblemService** — база задач (создание, редактирование).
- **TestingService** — приём ответов и автоматическая проверка.
- **ResultService** — статистика и прогресс.
- **ChatService** — хранение и выдача истории сообщений.
- **Realtime chat runtime** — WebSocket для чата в реальном времени.
- **Repositories** — слой доступа к данным.

Каждый сервис соответствует одному use case (сценарию использования) и реализует бизнес-логику независимо от HTTP-контекста.

3.4 Проектирование базы данных

Спроектирована реляционная база данных, охватывающая все ключевые сущности предметной области: пользователей, учебные классы, уроки, домашние задания, задачи, результаты тестирования и сообщения чата. ER-диаграмма данных представлена на рисунке 2.

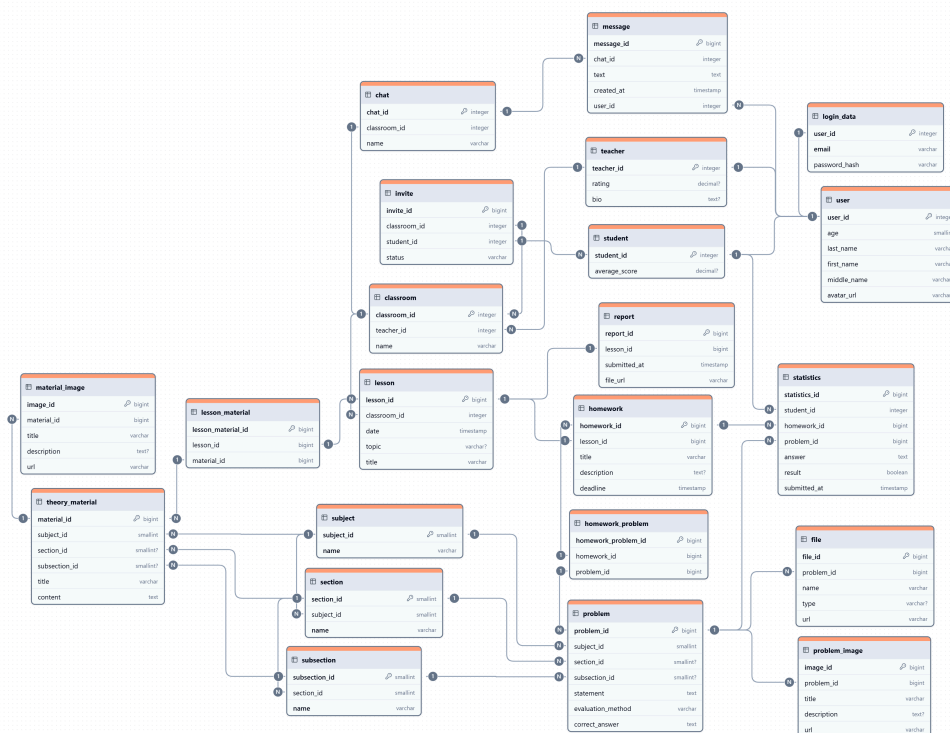


Рисунок 2 – ER-диаграмма данных системы управления обучением

Основные сущности базы данных включают:

- **user** — центральная сущность системы, хранящая общие данные пользователя (имя, email, аватар).
- **login_data** — изолированные данные аутентификации (хеш пароля Argon2).
- **teacher** и **student** — специализированные сущности с дополнительными атрибутами для соответствующих ролей.
- **class_group** — учебный класс, объединяющий студентов под руководством преподавателя.

- **lesson** — урок в рамках класса (тема, порядковый номер, статус публикации).
- **homework** — домашнее задание, привязанное к уроку.
- **problem** — задача, которая может быть использована в разных домашних заданиях (связь "многие ко многим").
- **statistics** — результат выполнения задачи студентом (ответ, статус проверки, количество попыток).
- **chat** и **message** — чат класса и сообщения.
- **invite** — приглашение в класс.
- **themes** — иерархия тем для структурирования учебного контента.

Реализация ORM-моделей выполнена с использованием SQLAlchemy с асинхронным драйвером `asynctrpg`. Пример модели пользователя приведён в листинге 2.

Листинг 2 – ORM-модель пользователя (SQLAlchemy)

```
# app/models/users.py
from sqlalchemy import Column, String, Boolean, DateTime
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import relationship
from app.db.session import Base
import uuid
from datetime import datetime, timezone

class User(Base):
    __tablename__ = "user"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    username = Column(String(50), unique=True, nullable=False, index=True)
    email = Column(String(255), unique=True, nullable=False, index=True)
    first_name = Column(String(100), nullable=False)
    last_name = Column(String(100), nullable=False)
    avatar_url = Column(String(500), nullable=True)
    is_active = Column(Boolean, default=True)
    created_at = Column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
    )
```

3.5 Выводы по главе

В данной главе разработана детальная архитектура серверной части системы управления обучением на основе Clean Architecture. Описаны четыре слоя архитектуры: API, Service, Repository и Models, их зоны ответственности и взаимодействие. Приведена структура директорий проекта, непосредственно отражающая выбранный архитектурный паттерн.

Описана реализация Dependency Injection как механизма связывания слоёв, обеспечивающего слабую связанность и тестируемость компонентов. Приведена диаграмма компонентов, отражающая распределение ответственности между модулями системы.

Спроектирована реляционная база данных, охватывающая все ключевые сущности предметной области. ER-диаграмма и пример ORM-модели демонстрируют применение Clean Architecture на уровне работы с данными. Разработанная архитектура является основой для реализации серверной части в следующей главе.

4 Разработка серверной части интернет-ресурса

4.1 Реализация REST API на FastAPI

Серверная часть системы реализована на языке Python 3.12+ с использованием веб-фреймворка FastAPI, обеспечивающего асинхронную обработку запросов, автоматическую валидацию данных через Pydantic v2 [24] и генерацию OpenAPI-спецификации [14,15]. При проектировании API использованы стандартные подходы REST, описанные в специализированной литературе [10]. Архитектура серверной части полностью соответствует Clean Architecture, описанной в предыдущей главе.

REST API предоставляет следующие группы эндпоинтов, сгруппированные по функциональным областям:

- **Auth** — регистрация, аутентификация, получение профиля текущего пользователя (/api/v1/auth/register, /login, /me).
- **Classrooms** — управление классами: создание, получение списка, вступление по приглашению (/api/v1/classes/).
- **Lessons** — управление уроками: создание, публикация, получение содержимого (/api/v1/lessons/).
- **Homework** — домашние задания: создание, получение состава задач (/api/v1/homework/).
- **Problems** — база задач: создание, редактирование (/api/v1/problems/).
- **Testing** — приём ответов на задачи: отправка ответа, получение результата (/api/v1/testing/).
- **Statistics** — статистика выполнения: прогресс студента, результаты по классу (/api/v1/statistics/).
- **Theory** — теоретические материалы (/api/v1/theory/).
- **Chat** — сообщения чата класса (/api/v1/chat/).

При разработке эндпоинтов соблюдается принцип разделения ответственности: каждый эндпоинт делегирует выполнение бизнес-операции соответствующему сервису и не содержит бизнес-логики непосредственно.

Пример реализации эндпоинта аутентификации приведён в листинге 3.

Листинг 3 – Реализация REST-эндпоинтов аутентификации

```

# app/api/v1/auth.py
import fastapi
from fastapi import status as http_status
from app.api import dependencies as deps
from app.models import users as user_models
from app.schemas import users as user_schemas
from app.services import auth as auth_service_module

router = fastapi.APIRouter()

@router.post(
    "/register",
    response_model=user_schemas.UserResponse,
    status_code=http_status.HTTP_201_CREATED,
)
async def register(
    user_data: user_schemas.UserCreate,
    auth_service: auth_service_module.AuthService =
        fastapi.Depends(deps.get_auth_service),
) -> user_schemas.UserResponse:
    """Регистрация нового пользователя."""
    return await auth_service.register_user(user_data)

@router.post("/login", response_model=user_schemas.TokenResponse)
async def login(
    credentials: user_schemas.LoginRequest,
    auth_service: auth_service_module.AuthService =
        fastapi.Depends(deps.get_auth_service),
) -> user_schemas.TokenResponse:
    """Аутентификация, возврат JWT-токена."""
    token = await auth_service.authenticate(credentials)
    if not token:
        raise fastapi.HTTPException(
            status_code=http_status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials",
        )
    return token

```

Каждый эндпоинт использует Dependency Injection для получения экземпляра сервиса, что обеспечивает тестируемость и слабую связанность.

4.2 Реализация сервисного слоя

Сервисный слой (Service Layer) содержит бизнес-логику приложения. Каждый сервис реализует определённый сценарий использования (use case)

и использует репозитории для доступа к данным. Сервисы не имеют прямых зависимостей от HTTP-контекста или FastAPI, что позволяет тестировать их изолированно.

Пример реализации сервиса аутентификации представлен в листинге 4. Сервис инкапсулирует логику регистрации, проверки существования email, хеширования пароля и создания JWT-токена.

Листинг 4 – Реализация сервиса аутентификации

```
from app.repositories import user as user_repository
from app.schemas import users as user_schemas
```

```
class AuthService:
```

```
    """Сервис аутентификации и управления пользователями."""
```

```
    def __init__(self, db: AsyncSession):
```

```
        self.db = db
```

```
        self.user_repo = user_repository.UserRepository(db)
```

```
        self.login_repo = user_repository.LoginDataRepository(db)
```

```
    async def register_user(
```

```
        self, user_data: user_schemas.UserCreate
```

```
) -> user_schemas.UserResponse:
```

```
    """Регистрация нового пользователя."""
```

```
    existing = await self.user_repo.get_by_email(user_data.email)
```

```
    if existing:
```

```
        raise domain_errors.BadRequestError(
```

```
            "Email already registered"
```

```
        )
```

```
    # Создание пользователя и login_data
```

```
    user = await self.user_repo.create({...})
```

```
    hashed = core_security.get_password_hash(user_data.password)
```

```
    await self.login_repo.create({
```

```
        "user_id": user.id,
```

```
        "password_hash": hashed,
```

```
    })
```

```
    return user_schemas.UserResponse.model_validate(user)
```

```
    async def authenticate(
```

```
        self, credentials: user_schemas.LoginRequest
```

```
) -> user_schemas.TokenResponse | None:
```

```
    """Аутентификация по email/username и паролю."""
```

```
    user = await self.user_repo.get_by_email_or_username(
```

```
        credentials.username_or_email
```

```
    )
```

```
    if not user:
```

```
        return None
```

```
    login_data = await self.login_repo.get_by_user_id(user.id)
```

```
    if not login_data:
```

```
        return None
```

```
    if not core_security.verify_password(
```

```
        credentials.password, login_data.password_hash
```

```
    ):
```

```
        return None
```

```
    # Генерация JWT
```

```
    token = core_security.create_access_token({
```

```
        "sub": str(user.id),
```

```
        "role": user.role,
```

```
    })
```

Данный пример демонстрирует ключевые принципы Clean Architecture в сервисном слое: сервис не импортирует ничего из FastAPI, использует репозитории для доступа к данным и возвращает DTO (UserResponse, TokenResponse).

4.3 Реализация слоя репозиториев

Слой репозиториев реализует паттерн Repository, изолируя логику SQL-запросов от сервисного слоя. Базовый репозиторий предоставляет общие CRUD-операции, а специализированные репозитории расширяют его доменными методами.

Листинг 5 – Реализация базового репозитория

```

# app/repositories/base.py
import typing as tp
import sqlalchemy as sa
from sqlalchemy.ext import asyncio as sa_asyncio
from app.db import session as db_session

T = tp.TypeVar("T", bound=db_session.Base)

class BaseRepository(tp.Generic[T]):
    """Базовый репозиторий с CRUD-операциями."""

    def __init__(self, model: type[T], db: sa_asyncio.AsyncSession):
        self.model = model
        self.db = db

    async def get_by_id(self, id: int) -> T | None:
        stmt = sa.select(self.model).where(
            self.model.id == id
        )
        result = await self.db.execute(stmt)
        return result.scalar_one_or_none()

    async def create(self, obj_in: dict[str, tp.Any]) -> T:
        db_obj = self.model(**obj_in)
        self.db.add(db_obj)
        await self.db.commit()
        await self.db.refresh(db_obj)
        return db_obj

    async def update(
        self, id: int, obj_in: dict[str, tp.Any]
    ) -> T | None:
        db_obj = await self.get_by_id(id)
        if not db_obj:
            return None
        for field, value in obj_in.items():
            if hasattr(db_obj, field):
                setattr(db_obj, field, value)
        await self.db.commit()
        await self.db.refresh(db_obj)
        return db_obj

```

Базовый репозиторий параметризован типом модели (`tp.Generic[T]`), что обеспечивает типобезопасность при создании специализированных

репозитория. Пример специализированного репозитория пользователя приведён в листинге 6.

Листинг 6 – Специализированный репозиторий пользователя

```
# app/repositories/user.py
from app.repositories.base import BaseRepository
from app.models.users import User, LoginData

class UserRepository(BaseRepository[User]):
    """Репозиторий для работы с пользователями."""

    def __init__(self, db):
        super().__init__(User, db)

    async def get_by_email(self, email: str) -> User | None:
        import sqlalchemy as sa
        stmt = sa.select(User).where(User.email == email)
        result = await self.db.execute(stmt)
        return result.scalar_one_or_none()

    async def get_by_email_or_username(
        self, login: str
    ) -> User | None:
        import sqlalchemy as sa
        stmt = sa.select(User).where(
            sa.or_(User.email == login, User.username == login)
        )
        result = await self.db.execute(stmt)
        return result.scalar_one_or_none()
```

4.4 Реализация слоя безопасности

Безопасность серверной части реализована на двух уровнях: хеширование паролей с использованием алгоритма Argon2id и аутентификация через JSON Web Token (JWT).

Argon2id является современным стандартом хеширования паролей, устойчивым к атакам с использованием GPU и ASIC. Данный алгоритм рекомендован в качестве основного для защиты учётных данных веб-приложений [25]. Реализация хеширования приведена в листинге 7.

Листинг 7 – Реализация безопасности: Argon2id + JWT

```

# app/core/security.py
import argon2
import jose.jwt as jose_jwt
from app.core import config as core_config
from app.core import datetime_extensions as dte

# Argon2id с рекомендуемыми параметрами
ph = argon2.PasswordHasher(
    time_cost=2,          # Количество итераций
    memory_cost=65536,    # Память 64 MB
    parallelism=1,        # Поток
    hash_len=32,          # Длина хеша
    salt_len=16,          # Длина соли
)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Верификация пароля через Argon2id."""
    try:
        ph.verify(hashed_password, plain_password)
        return True
    except argon2.exceptions.VerifyMismatchError:
        return False

def create_access_token(data: dict) -> str:
    """Создание JWT-токена с expiration."""
    to_encode = data.copy()
    expire = dte.now_utc() + core_config.settings.ACCESS_TOKEN_EXPIRE
    to_encode.update({"exp": expire})
    return jose_jwt.encode(
        to_encode,
        core_config.settings.SECRET_KEY,
        algorithm=core_config.settings.ALGORITHM,
    )

```

4.5 Конфигурация и запуск приложения

Конфигурация приложения осуществляется через переменные окружения с использованием библиотеки pydantic-settings. Основные параметры включают подключение к базе данных, настройки CORS, секретный ключ для JWT и параметры Redis (опционально, для realtime-чата). Листинг 8 – Конфигурация приложения через pydantic-settings


```

# app/core/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    APP_NAME: str = "Web Education Platform API"
    APP_VERSION: str = "1.0.0"
    DEBUG: bool = True
    API_V1_PREFIX: str = "/api/v1"
    DATABASE_URL: str = "postgresql://..."
    REDIS_URL: str | None = None # Опционально
    SECRET_KEY: str = "change-me"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 30
    BACKEND_CORS_ORIGINS: list[str] = [
        "http://localhost:5173"
    ]

    model_config = {"env_file": ".env"}

settings = Settings()

```

Точка входа приложения (листинг 9) инициализирует FastAPI, подключает middleware (CORS, RequestIdMiddleware), регистрирует обработчики доменных ошибок и монтирует API-роутер с префиксом /api/v1.

Листинг 9 – Точка входа приложения FastAPI

```
# app/main.py (ключевые элементы)
import fastapi
from fastapi.middleware import cors as fastapi_cors
from app.api import errors as api_errors
from app.api import openapi as api_openapi
from app.api.middleware import request_id as request_id_middleware
from app.api import v1 as api_v1
from app.core import config as core_config

app = fastapi.FastAPI(
    title=core_config.settings.APP_NAME,
    version=core_config.settings.APP_VERSION,
    debug=core_config.settings.DEBUG,
    docs_url="/api/docs",
    openapi_url="/api/openapi.json",
)

app.add_middleware(fastapi_cors.CORSMiddleware, ...)
app.add_middleware(request_id_middleware.RequestIdMiddleware)
api_errors.register_exception_handlers(app)
api_openapi.install_openapi_patch(app)
app.include_router(
    api_v1.api_router,
    prefix=core_config.settings.API_V1_PREFIX,
)
```

4.6 Интеграция с realtime-чатом

В дополнение к REST API серверная часть включает поддержку WebSocket-чата для обмена сообщениями в реальном времени. Механизм realtime-чата реализован с использованием Connection Manager и опционального Redis Pub/Sub для поддержки многоэкземплярного развёртывания:

```
# app/main.py (startup)
from app.realtime import connection_manager as cm
from app.realtime import redis_pubsub as rp

@app.on_event("startup")
async def on_startup():
    app.state.chat_connection_manager = cm.ConnectionManager()
    if core_config.settings.REDIS_URL:
        redis_client = redis_asyncio.from_url(
```

```

        core_config.settings.REDIS_URL
    )
    app.state.chat_broker = rp.RedisPubSubBroker(
        redis_client,
        manager=app.state.chat_connection_manager,
    )

```

Redis является опциональным компонентом: если REDIS_URL не задан, приложение работает без realtime-функций, что упрощает локальную разработку.

4.7 Выводы по главе

В данной главе описана реализация серверной части системы управления обучением на основе спроектированной Clean Architecture. Разработан REST API на FastAPI, включающий endpoints для аутентификации, управления классами, уроками, домашними заданиями, тестированием и чатом. Каждый endpoint следует принципу разделения ответственности, делегируя бизнес-логику сервисам.

Реализован сервисный слой с примерами аутентификации и управления пользователями. Сервисы не зависят от HTTP-контекста и могут тестироваться изолированно. Слой репозитория, основанный на базовом CRUD-репозитории, изолирует SQL-запросы от бизнес-логики.

Обеспечена безопасность приложения: хеширование паролей через Argon2id и аутентификация через JWT. Реализована конфигурация через переменные окружения (pydantic-settings) и опциональная поддержка realtime-чата через Redis Pub/Sub.

Разработанная серверная часть обеспечивает полный цикл операций, необходимых для функционирования системы управления обучением, и полностью соответствует принципам Clean Architecture.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана серверная часть веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture.

На первом этапе выполнен анализ предметной области — систем управления обучением (LMS). Определены функциональные и нефункциональные требования к серверной части, сформирована матрица отслеживаемости аналогов LMS-систем, включающая сравнение платформ Moodle, Canvas LMS, Google Classroom и разрабатываемой системы. Анализ показал, что ключевыми преимуществами разрабатываемого решения являются лёгкая архитектура, соответствие современным стандартам REST API и использование актуального технологического стека.

Проведён обзор и сравнительный анализ архитектурных паттерновDDD, Clean Architecture и MVC. На основе критериев сложности, тестируемости, независимости от фреймворков и применимости в условиях ограниченных ресурсов обоснован выбор Clean Architecture как основного архитектурного подхода. Данный паттерн обеспечивает чёткое разделение ответственности между слоями приложения, независимость бизнес-логики от внешних фреймворков и высокую тестируемость компонентов.

Разработана детальная архитектура серверной части, включающая четыре слоя: API (уровень представления), Service (уровень бизнес-логики), Repository (уровень доступа к данным) и Models (уровень ORM). Архитектура реализована с использованием принципов Dependency Injection, что обеспечивает слабую связанность между слоями и возможность независимого тестирования каждого из них.

Реализована серверная часть на языке Python 3.12+ с использованием фреймворка FastAPI, СУБД PostgreSQL и библиотеки SQLAlchemy. Разработан REST API, включающий эндпоинты для аутентификации, управления классами, уроками, домашними заданиями, тестированием и чатом. Создана схема базы данных, охватывающая таких сущности, как пользователи, классы, уроки, домашние задания, задачи, результаты тестирования и сообщения чата.

В процессе реализации использованы паттерны Repository, Service Layer и DTO, соответствующие Clean Architecture.

Корректность разработанной архитектуры подтверждена реализованным прототипом системы, включающим как серверную, так и клиентскую части. Серверная часть обеспечивает полный цикл операций: от регистрации пользователя и управления курсами до проведения тестирования и получения статистики. Подготовленная инфраструктура на базе Docker (PostgreSQL, Redis) обеспечивает воспроизводимость окружения.

Полученные результаты подтверждают применимость Clean Architecture и выбранного технологического стека для построения серверной части системы управления обучением. Разработанная архитектура может служить основой для дальнейшего расширения функциональности, интеграции с внешними сервисами и развития в направлении полноценной образовательной платформы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Минакова О.В. Программная инженерия: Основные принципы, методы и инструменты [Электронный ресурс]: учебник для вузов ISBN 978–5–507–49278–7. Санкт-Петербург: Лань, 2024. 212 с. URL: <https://e.lanbook.com/book/414989> (дата обращения: 06.12.2025).
2. Rosário A.T., Dias J.C. Learning Management Systems in Education: Research and Challenges: DOI: 10.4018/978-1-6684-4706-2.ch003 // Digital Active Methodologies for Educative Learning Management / под ред. Geada N., Jamil G.L. Hershey, PA: IGI Global, 2022. Сс. 47–77.
3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения: ISBN 978–5–4461–1903–1. 2-е изд. Санкт-Петербург: Питер, 2021. 352 с.
4. Киселева Т.В. Программная инженерия [Электронный ресурс]: учебное пособие. Ставрополь: СКФУ, 2017. Т. 2. 100 с. URL: <https://e.lanbook.com/book/155149> (дата обращения: 30.11.2025).
5. Аналитический Центр Москвы. Спецпроект о школьниках Москвы [Электронный ресурс]. Москва, 2025. URL: <https://ac.mos.ru/special-projects/moscow-school-students/> (дата обращения: 21.12.2025).
6. Туманова М.Б., Михайлова Е.К., Муравьева Е.А. Проектирование программных систем [Электронный ресурс]: учебное пособие ISBN 978–5–7339–2050–4. Москва: РТУ МИРЭА, 2023. 138 с. URL: <https://e.lanbook.com/book/398273> (дата обращения: 29.11.2025).
7. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. Москва: Стандартинформ, 2017.
8. Баланов А.Н. Комплексное руководство по разработке: от мобильных приложений до веб-технологий [Электронный ресурс]: учебное пособие для вузов ISBN 978–5–507–53193–6. 2-е изд. Санкт-Петербург: Лань, 2025. 412 с. URL: <https://e.lanbook.com/book/478178> (дата обращения: 07.12.2025).
9. Docker Compose documentation [Электронный ресурс]. URL: <https://docs.docker.com/compose/> (дата обращения: 11.05.2026).

10. Алпатов А.Н. Интерфейсы прикладного программирования [Электронный ресурс]: учебное пособие ISBN 978–5–7339–2342–0. Москва: РТУ МИРЭА, 2024. 157 с. URL: <https://e.lanbook.com/book/457043> (дата обращения: 07.12.2025).
11. Тузовский А.Ф. Проектирование и разработка web-приложений: учебное пособие для вузов ISBN 978–5–534–515–8. Москва: Юрайт, 2021. 218 с.
12. Персиваль Г., Грегори Б. Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура: ISBN 978–5–4461–1969–7. Санкт-Петербург: Питер, 2022. 336 с.
13. Python 3.12 Documentation [Электронный ресурс]. URL: <https://docs.python.org/3.12/> (дата обращения: 11.05.2026).
14. Елисеев А.И., Минин Ю.В. Разработка программных интерфейсов веб-приложений с использованием фреймворка FastAPI [Электронный ресурс]: учебное пособие ISBN 978–5–8265–2821–1. Тамбов: ТГТУ, 2024. 81 с. URL: <https://e.lanbook.com/book/472310> (дата обращения: 07.12.2025).
15. Ramírez S. FastAPI — официальная документация [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 11.05.2026).
16. Stack Overflow Developer Survey 2025 [Электронный ресурс]. 2025. URL: <https://survey.stackoverflow.co/2025/technology#1-web-frameworks-and-technologies> (дата обращения: 07.12.2025).
17. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 11.05.2026).
18. Волк В.К., Осеев В.Ю., Черепанов О.С. Базы данных [Электронный ресурс]: учебник ISBN 978–5–9729–2594–0. Вологда: Инфра-Инженерия, 2025. 544 с. URL: <https://e.lanbook.com/book/499772> (дата обращения: 07.12.2025).
19. SQLAlchemy 2.0 Documentation [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 11.05.2026).
20. Alembic — официальная документация [Электронный ресурс]. URL: <https://alembic.sqlalchemy.org/> (дата обращения: 11.05.2026).
21. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 11.05.2026).

22. Feature-Sliced Design: официальная документация [Электронный ресурс]. URL: <https://feature-sliced.github.io/documentation/ru/> (дата обращения: 06.12.2025).
23. Водяхо А.И. и др. Архитектурные решения информационных систем [Электронный ресурс]: учебник для вузов ISBN 978–5–507–44710–7. 3-е изд. Санкт-Петербург: Лань, 2022. 356 с. URL: <https://e.lanbook.com/book/254624> (дата обращения: 07.12.2025).
24. Pydantic V2 Documentation [Электронный ресурс]. URL: <https://docs.pydantic.dev/latest/> (дата обращения: 11.05.2026).
25. Хоффман Э. Безопасность веб-приложений: ISBN 978–5–4461–1706–8. Санкт-Петербург: Питер, 2021. 336 с.